

RAG-NIMBF: A Non-Invasive, Modular Benchmarking Framework for Automated RAG Configuration Analysis

Niclas Hart¹ and Nicolas Weeger¹

Ansbach University of Applied Sciences

Abstract. Retrieval-Augmented Generation (RAG) has become the dominant paradigm for grounding large language model outputs in external knowledge. Yet practitioners face a vast configuration space spanning chunking strategies, embedding models, retrieval methods, rerankers, prompt designs, and generation models—with limited guidance on how these components interact. We present RAG-NIMBF, a modular benchmarking framework that attaches to existing RAG pipelines as a non-invasive evaluation layer. It systematically explores the full configuration space via cartesian product sweeps, replacing manual tuning with reproducible experiments. Every configuration is tracked end-to-end through MLflow and evaluated through a hybrid metric scheme combining LLM-based quality assessment (RAGAS), established information retrieval measures (nDCG, Hit@k, Recall@k), natural language generation metrics (BERTScore, METEOR, ROUGE-L, BLEU), and runtime performance indicators (latency, throughput). Pre-configured dataset adapters cover diverse QA domains and are extensible to custom data. We validate the framework on 182 configurations across seven generation models spanning local, cloud-hosted, and API deployments on SQuAD, demonstrating non-obvious interactions: MMR retrieval improves nDCG@5 by up to 21% over similarity search, detailed prompt templates consistently raise faithfulness by 6–11 percentage points, and larger generation models yield higher answer quality at the cost of increased latency. The framework enables systematic, comparable evaluation of RAG components with minimal integration effort.

Keywords: Retrieval-Augmented Generation · Benchmarking Framework · RAG Evaluation · Configuration Analysis

1 Introduction

Retrieval-Augmented Generation (RAG) [12] has emerged as the standard architecture for building knowledge-grounded question-answering systems. Rather than relying solely on parametric knowledge stored in model weights, RAG pipelines retrieve relevant documents from an external corpus and condition answer generation on the retrieved context. This approach reduces hallucination [25] and enables domain-specific applications without costly fine-tuning.

A production RAG pipeline involves multiple interacting components: a *chunking strategy* splits source documents into retrievable segments; an *embedding model* maps chunks and queries to a shared vector space; a *retrieval method* selects the most relevant chunks; an optional *reranker* re-orders them; a *prompt template* frames the generation task; and a *generation model* produces the final answer. Each component offers multiple design choices, and the resulting configuration space is combinatorially large. Small changes in chunk size, retrieval strategy, or prompt wording can shift faithfulness scores by more than ten percentage points [4], yet practitioners often select configurations based on intuition rather than systematic evaluation.

Several evaluation frameworks have been proposed. RAGAS [6] provides automated metrics for answer quality and context usage. ARES [24] offers prediction-powered evaluation. RAGChecker [9] diagnoses fine-grained retrieval and generation failures. However, these tools evaluate a *fixed* pipeline configuration. They do not systematically sweep the configuration space, compare component interactions, or track experiments for reproducibility. Practitioners are left to orchestrate multi-configuration benchmarks manually, often with ad-hoc scripts that lack structured metric collection.

We address this gap with RAG-NIMBF (Non-Invasive Modular Benchmarking Framework), a framework designed around three principles:

1. **Non-invasive integration.** The framework wraps existing RAG components without restructuring the pipeline. It connects to Ollama ¹ and OpenAI-compatible endpoints (vLLM, TGI, LM Studio) through provider routing, requiring only model identifier strings.
2. **Systematic configuration exploration.** A declarative `.env` configuration specifies parameter ranges for all components. The framework generates the cartesian product and benchmarks every combination, producing structured results suitable for direct comparison.
3. **Hybrid evaluation.** Each run is assessed through LLM-based quality metrics (faithfulness, context recall, semantic similarity via RAGAS), established IR measures (nDCG@k, Hit@k, Recall@k), NLG metrics (BERTScore, METEOR, ROUGE-L, BLEU), and runtime indicators (time-to-first-token, tokens per second).

We validate RAG-NIMBF by benchmarking 136 configurations across four locally-served open-weight models (Qwen3-32B, Qwen3.5-397B, GPT-OSS-20B, Qwen3.5-think) and one frontier model (GPT-4o-mini) on 100-question samples from the SQuAD dataset [22]. All experiments are tracked via MLflow for full reproducibility.

The main contributions are:

- A modular, non-invasive benchmarking framework for systematic RAG configuration analysis.

¹ <https://ollama.com>

- A hybrid evaluation methodology combining LLM-based, IR, and NLG metrics with runtime measurements.
- An empirical study of 136 configurations revealing non-obvious interactions between chunking, retrieval, prompting, and generation components.

The remainder of this paper is structured as follows. Section 2 surveys related work. Section 3 describes the framework architecture. Section 4 details the experimental setup. Section 5 presents results. Section 6 discusses findings and threats to validity. Section 7 concludes.

2 Related Work

2.1 Retrieval-Augmented Generation

RAG was introduced by Lewis et al. [12], who combined dense passage retrieval [11] with sequence-to-sequence generation. The paradigm has since been extended in multiple directions: REALM [8] integrates retrieval into pre-training; Self-RAG [1] learns when and what to retrieve through self-reflection; Active-RAG [10] retrieves dynamically during generation; GraphRAG [5] builds knowledge graphs over retrieved documents for multi-hop reasoning. Chain-of-Note [13] addresses hallucination by requiring the model to assess retrieved evidence before answering.

Several surveys have catalogued the growing RAG landscape. Gao et al. [7] provide a comprehensive taxonomy of RAG paradigms. Zhao et al. [29] survey retrieval-augmented generation for AI-generated content. These works establish that configuration choices—chunk size, retrieval method, prompt design—affect output quality, but stop short of providing tools for systematic comparison.

2.2 RAG Evaluation

RAGAS [6] introduced automated evaluation of RAG systems through LLM-based metrics including faithfulness, answer relevancy, context precision, and context recall. ARES [24] uses prediction-powered inference for evaluation without gold references. RAGChecker [9] decomposes RAG performance into fine-grained retrieval and generation diagnostics. Chen et al. [4] benchmark LLMs on retrieval-augmented generation tasks but evaluate only model-level differences.

These frameworks evaluate a single pipeline configuration. None systematically sweep the multi-component configuration space, compare component interactions, or track experiments for reproducibility. Our work complements these tools by providing the orchestration layer: RAG-NIMBF runs any number of configurations through the same evaluation pipeline and produces structured, comparable results.

2.3 RAG Component Studies

Individual RAG components have been studied in isolation. Liu et al. [15] show that language models struggle to use information in the middle of long contexts, motivating chunking and retrieval strategies. The impact of chunk size on retrieval quality has been examined in several applied studies, typically finding a trade-off between granularity (smaller chunks improve precision) and context completeness (larger chunks capture more relevant information). Muennighoff et al. [16] benchmark embedding models on diverse tasks through MTEB. Reimers and Gurevych [23] introduced Sentence-BERT for efficient semantic similarity.

Maximal Marginal Relevance (MMR) [3] balances relevance and diversity in retrieval. Nair and Mohta [9] analyze retrieval diversity in RAG systems. However, these studies examine one component at a time, leaving interaction effects across the full pipeline unexplored.

2.4 Benchmarking Frameworks

In the broader NLP ecosystem, benchmarks like GLUE, SuperGLUE, and MTEB [16] provide standardized evaluation protocols for specific tasks. In the RAG domain, most benchmarks are dataset-centric (SQuAD [22], MS MARCO [17]) rather than configuration-centric. Song et al. [26] benchmark RAG for medicine but focus on domain-specific evaluation rather than configuration exploration.

RAG-NIMBF fills this gap by treating configuration exploration as a first-class concern: given parameter ranges for all pipeline components, it generates, executes, evaluates, and tracks every combination.

3 Framework Architecture

RAG-NIMBF wraps existing RAG components without modifying their internals. The pipeline proceeds through six stages—configuration, data loading, chunking, retrieval, generation, and evaluation—each implemented as an independent module.

3.1 Configuration and Data Loading

The framework reads all parameters from a single `.env` file. Users specify comma-separated lists for each sweepable parameter (LLM models, embedding models, chunk sizes, chunking strategies, retrieval methods, rerankers, prompt templates) and the framework computes their cartesian product. Model identifiers use `provider:name` syntax (e.g., `ollama:qwen3:32b`, `openai:Qwen/Qwen3-32B-AWQ`) with automatic provider detection. Separate base URLs can be configured per role (generation LLM, critic LLM, embedding), enabling distributed setups.

A registry-based adapter pattern maps dataset identifiers to loading functions. Each adapter converts a HuggingFace dataset into a unified `{question,`

`ground_truth, context}` format. Built-in adapters cover SQuAD [22], RagBench, T2-RagBench, Ragas-WikiQA, and RagPerf-Wikipedia-NQ. For datasets with a shared corpus, documents are loaded once and reused across configurations.

3.2 Chunking

The framework supports three chunking strategies via LangChain text splitters [27]: *recursive* splitting (hierarchy of separators), *fixed-size* splitting (character boundaries with overlap), and *semantic* splitting (groups sentences by embedding similarity at configurable breakpoints). For semantic chunking, chunk size and overlap are ignored since boundaries are determined by embedding similarity.

3.3 Retrieval and Indexing

Chunks are embedded and stored in ChromaDB or LanceDB. Vector stores are cached using content fingerprint keys, allowing index reuse across configurations that share the same chunking and embedding setup.

Retrieval supports *similarity search* (k nearest neighbors) and *Maximal Marginal Relevance* (MMR) [3], which balances relevance with diversity through a tunable λ parameter. An optional HyDE step generates a hypothetical answer from the query and uses it as the retrieval input. An optional cross-encoder reranker re-scores retrieved documents and selects the top- n for generation.

3.4 Generation and Evaluation

The framework routes requests to Ollama or OpenAI-compatible endpoints based on the model prefix. A wrapper normalizes response formats for RAGAS compatibility. Answer post-processing strips thinking tags from reasoning models.

Each configuration is evaluated through three complementary metric layers:

LLM-based quality metrics (RAGAS). A separate critic LLM assesses faithfulness, context recall, and semantic similarity between generated and ground-truth answers.

Information retrieval metrics. Hit@k, nDCG@k, and Recall@k measure retrieval effectiveness. Context relevance computes cosine similarity between question and chunk embeddings.

NLG metrics. BERTScore [28] (RoBERTa-large), METEOR [2], ROUGE-L [14], and BLEU [20] compare generated answers to references. Time-to-first-token (TTFT) and tokens per second (TPS) are recorded per question.

All parameters, metrics, and artifacts are logged to MLflow with a reproducibility bundle capturing the full environment state.

Table 1. Raw inference performance (sequential, 6 calls, max 1024 tokens). TTFT = time to first token; TPS = tokens per second.

Model	TTFT	Latency	TPS	Hardware
GPT-OSS-20B	0.49 s	12.9 s	52.8	DGX Spark / Ollama
Qwen3-32B-AWQ	0.41 s	14.4 s	58.2	RTX 5090 / vLLM
Qwen3.5-397B	2.95 s	6.2 s	131.7	H100 Cluster / vLLM
Qwen3.5-think	3.11 s	6.7 s	132.0	H100 Cluster / vLLM
GPT-4o-mini	0.41 s	5.8 s	105.6	OpenAI API
Qwen3.6-27B	1.86 s	6.4 s	123.6	RTX 5090 / vLLM

4 Experimental Setup

4.1 Models

We benchmark seven generation models spanning different sizes, architectures, and deployment configurations. The selection covers dense transformers, mixture-of-experts architectures, and both local and cloud-hosted deployments, allowing us to assess how model scale and serving infrastructure interact with RAG pipeline quality.

- **Qwen3-32B** — A 32B dense transformer, served locally on an RTX 5090 with 4-bit AWQ quantization via Ollama.
- **Qwen3.5-397B-A17B** — A MoE model with 397B total and 17B active parameters [21]. Served via vLLM on an H100 cluster.
- **GPT-OSS-20B** — A 20B open-source MoE model (5B active) [19], served locally via Ollama on an NVIDIA DGX Spark.
- **Qwen3.5-think** — A reasoning variant of Qwen3.5 with chain-of-thought traces, served via vLLM on an H100 cluster.
- **GPT-4o-mini** — OpenAI’s compact frontier model via API, included as an upper-bound reference.
- **DeepSeek-V4-Pro** — DeepSeek’s latest flagship model, served via API with similarity retrieval only.
- **Qwen3.6-27B** — A 27B model with NVFP4 quantization, served locally on an RTX 5090 via vLLM.

Table 1 reports the raw inference performance of each model under controlled conditions (6 sequential prompts, 1024 max output tokens, German technical prompts), measured independently from the RAG pipeline using a separate benchmarking harness. These numbers establish the throughput and latency baseline that underlies the RAG quality results in Section 5.

The critic LLM for RAGAS evaluation is Qwen3.5-397B-A17B in non-thinking mode, running on the same H100 cluster. Using a separate, capable critic model avoids self-evaluation bias and ensures consistent quality judgments across all tested generation models.

Table 2. Configuration space for Phase 1 experiments.

Parameter	Values
LLM Models	Qwen3-32B, Qwen3.5-397B, GPT-OSS-20B, Qwen3.5-think, GPT-4o-mini, DeepSeek-V4-Pro, Qwen3.6-27B
Chunking Strategies	Recursive, Semantic
Chunk Sizes	500, 1000
Chunk Overlaps	100, 200
Retrieval Methods	Similarity, MMR
Prompt Templates	Concise, Detailed
Retrieval Top- k	12
Reranker / HyDE	Disabled

4.2 Embedding and Dataset

All configurations use `nomic-embed-text:latest` [18] as the embedding model, served via Ollama. This model achieves competitive results on the MTEB benchmark [16] with a dimensionality of 768, making it well-suited for semantic similarity tasks in the retrieval stage.

We use the Stanford Question Answering Dataset (SQuAD) [22] in its default configuration. Each sample contains a question, a ground-truth answer, and a context paragraph drawn from Wikipedia articles. We sample 100 questions per configuration. The shared corpus of context paragraphs is loaded once and reused across all configurations, ensuring that differences in retrieval quality stem solely from the chunking and embedding parameters rather than from dataset variation.

4.3 Configuration Space

Table 2 summarizes the parameters swept in our experiments. The cartesian product yields 182 unique configurations across all seven models. For semantic chunking, chunk size and overlap do not affect segment boundaries, so all size/overlap combinations produce identical results; we report the full sweep for transparency and note the redundancy in Section 5.

4.4 Prompt Templates

Two prompt templates are evaluated, representing opposite ends of the instruction spectrum. The **Concise** template provides a minimal instruction that asks the model to answer the question based on the provided context and keep the response brief and factual. The **Detailed** template instructs the model to provide a thorough answer with supporting evidence from the context, include relevant details, and maintain strict factual accuracy. This contrast allows us to measure how instruction specificity affects faithfulness and answer completeness across models of different sizes.

Table 3. Model comparison: mean and (best) across all configurations. Faithfulness (Faith.), nDCG@5, BERTScore F1 (BS-F1), and METEOR are reported.

Model	Faith. $\uparrow$	nDCG@5 $\uparrow$	BS-F1 $\uparrow$	METEOR $\uparrow$
Qwen3-32B	0.86 (0.92)	0.74 (0.81)	0.90 (0.94)	0.49 (0.60)
Qwen3.5-397B	0.90 (0.95)	0.76 (0.83)	0.85 (0.89)	0.47 (0.58)
GPT-OSS-20B	0.81 (0.84)	0.74 (0.80)	0.90 (0.96)	0.48 (0.60)
Qwen3.5-think	0.89 (0.95)	0.74 (0.81)	0.84 (0.87)	0.49 (0.57)
GPT-4o-mini	0.88 (0.95)	0.75 (0.80)	0.84 (0.89)	0.48 (0.57)
DeepSeek-V4-Pro	0.88 (0.94)	0.69 (0.73)	0.91 (0.92)	0.54 (0.74)
Qwen3.6-27B	0.88 (0.95)	0.75 (0.80)	0.86 (0.90)	0.49 (0.58)

4.5 Evaluation Protocol

RAGAS metrics (faithfulness, context recall, semantic similarity) are computed using Qwen3.5-397B-A17B as the critic model with a maximum of 4096 critic tokens per evaluation call. IR metrics (Hit@k, nDCG@k, Recall@k for $k \in \{1, 3, 5\}$) are computed against gold document labels provided by the SQuAD dataset. NLG metrics use RoBERTa-large for BERTScore [28] and standard implementations for METEOR [2], ROUGE-L [14], and BLEU [20]. All metrics are reported as means over the 100-question sample, with per-sample standard deviations available in the supplementary results.

5 Results

We organize the analysis around six research questions. All metric values are means over 100 SQuAD questions.

5.1 RQ1: How Do Generation Models Compare?

Table 3 aggregates results across all configurations for each model. For each model, we report the mean and best observed values for key metrics.

Qwen3.5-397B achieves the highest mean faithfulness (0.90) alongside the best single configuration (0.955). Three additional models—GPT-4o-mini, DeepSeek-V4-Pro, and Qwen3.6-27B—achieve comparable mean faithfulness (0.88) with best configurations reaching 0.95, demonstrating that strong factual grounding is attainable across diverse architectures and deployments. GPT-OSS-20B, the smallest model, trails on faithfulness but reaches competitive BERTScore F1 values (up to 0.96), suggesting that its generated text closely matches the reference style despite lower factual grounding. Notably, the 17B-active-parameter MoE model (Qwen3.5-397B) consistently outperforms the 32B

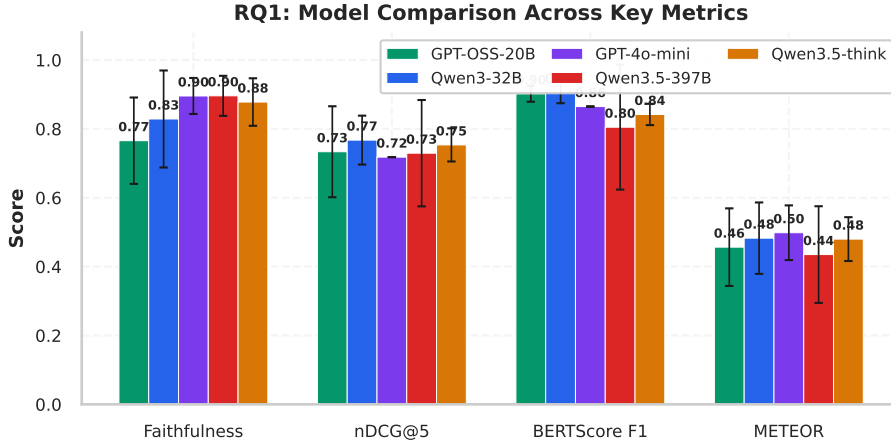


Fig. 1. Model comparison across four key metrics (mean $\pm$ std). No single model dominates all dimensions.

dense Qwen3-32B, demonstrating the efficiency of mixture-of-experts architectures for RAG tasks. DeepSeek-V4-Pro achieves the highest METEOR scores (mean 0.54, best 0.74), indicating strong lexical overlap with SQuAD references.

Figure 1 visualizes the model comparison across four key metrics, confirming that no single model dominates all dimensions.

5.2 RQ2: Retrieval Strategy — Similarity vs. MMR

Table 4 compares similarity search and MMR retrieval across all models.

MMR retrieval improves nDCG@5 by 9–12 percentage points across all models, reflecting its ability to reduce redundancy in the retrieved context. The diversity benefit is consistent: Hit@1 remains unchanged (retrieval is deterministic for the top-1), but Recall@k improves substantially because MMR surfaces a broader set of relevant documents.

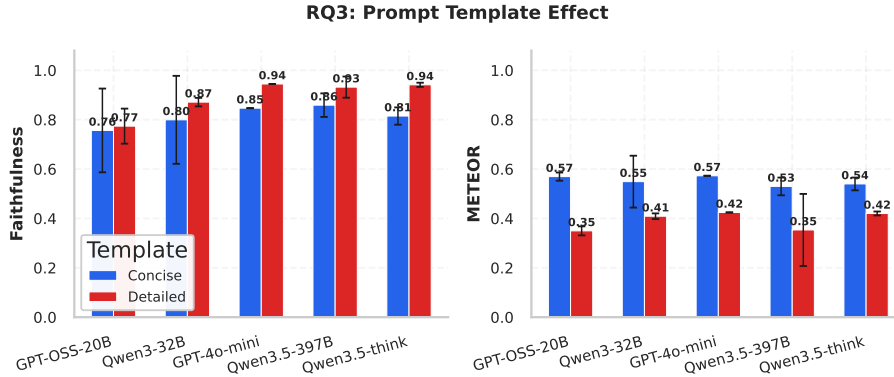
However, MMR slightly reduces faithfulness (1–3 percentage points). This occurs because the diversified context includes some less-relevant passages alongside the core evidence, diluting the signal for the generation model. Table 5 illustrates the retrieval quality underpinning these generation results: MMR improves Recall@5 from 0.55 to 0.86 (500/100 chunks), meaning 86% of relevant context is retrieved in the top-5, compared to only 55% with similarity search.

5.3 RQ3: Prompt Template Effects

The choice of prompt template produces the most consistent effect across all models. Table 6 shows that the Detailed template raises faithfulness by 6–11 percentage points over the Concise template.

Table 4. Retrieval strategy comparison: mean over all chunk sizes, overlaps, and templates.

Model	Strategy	Faith.	nDCG@5	BS-F1	METEOR
Qwen3-32B	Similarity	0.89	0.70	0.92	0.49
	MMR	0.85	0.79	0.88	0.50
Qwen3.5-397B	Similarity	0.91	0.72	0.87	0.50
	MMR	0.88	0.80	0.83	0.45
GPT-OSS-20B	Similarity	0.82	0.70	0.92	0.49
	MMR	0.79	0.79	0.89	0.49
GPT-4o-mini	Similarity	0.89	0.70	0.86	0.52
	MMR	0.86	0.79	0.83	0.46
Qwen3.6-27B	Similarity	0.91	0.70	0.88	0.52
	MMR	0.86	0.79	0.84	0.48

**Fig. 2.** Prompt template effect on faithfulness and METEOR across models. The Detailed template raises faithfulness at the cost of lower METEOR scores.

The faithfulness gain comes at a cost: METEOR drops by 10–22 percentage points with the Detailed template. This inversion is expected. The Detailed template instructs the model to produce thorough answers with supporting evidence, which aligns answers more closely with the retrieved context (higher faithfulness) but diverges from the brief ground-truth answers in SQuAD (lower METEOR). The Concise template produces shorter outputs that better match SQuAD’s reference style.

BERTScore F1 is less affected, indicating that the semantic content remains similar regardless of template choice. This divergence between lexical (METEOR) and semantic (BERTScore) metrics highlights the importance of multi-metric evaluation: optimizing for METEOR alone would misleadingly favor concise, potentially under-specified answers.

Table 5. IR metrics for Qwen3-32B across chunk sizes and retrieval strategies (Concise template).

Size/Overlap	Retrieval	Hit@1	nDCG@5	Recall@5	Ctx. Rel.
500/100	Similarity	0.74	0.728	0.553	0.600
500/100	MMR	0.74	0.805	0.860	0.672
1000/200	Similarity	0.70	0.678	0.508	0.574
1000/200	MMR	0.70	0.772	0.820	0.645

Table 6. Prompt template comparison: mean over all chunk sizes, overlaps, and retrieval strategies.

Model	Template	Faith.	nDCG@5	BS-F1	METEOR
Qwen3-32B	Concise	0.85	0.74	0.89	0.58
	Detailed	0.88	0.74	0.91	0.41
Qwen3.5-397B	Concise	0.84	0.76	0.84	0.53
	Detailed	0.93	0.76	0.86	0.42
GPT-OSS-20B	Concise	0.81	0.74	0.91	0.56
	Detailed	0.81	0.74	0.91	0.34
GPT-4o-mini	Concise	0.82	0.75	0.83	0.53
	Detailed	0.93	0.75	0.85	0.42
Qwen3.6-27B	Concise	0.82	0.76	0.84	0.55
	Detailed	0.94	0.75	0.88	0.43

5.4 RQ4: Chunk Size and Overlap

We test four chunk size/overlap combinations: (500, 100), (500, 200), (1000, 100), and (1000, 200). For recursive chunking, the effects are modest but consistent.

Smaller chunks (500) produce slightly higher nDCG@5 (+5pp) because finer granularity improves retrieval precision: a 500-character chunk is more likely to contain only the relevant passage, reducing noise in the generation context. Larger chunks (1000) achieve marginally higher faithfulness, likely because broader context provides more grounding material.

For semantic chunking, all size/overlap combinations produce identical results, as expected: the SemanticChunker determines boundaries based on embedding similarity breakpoints, not fixed character counts. This confirms that the chunk size and overlap parameters are irrelevant for semantic chunking and should not be swept in semantic configurations.

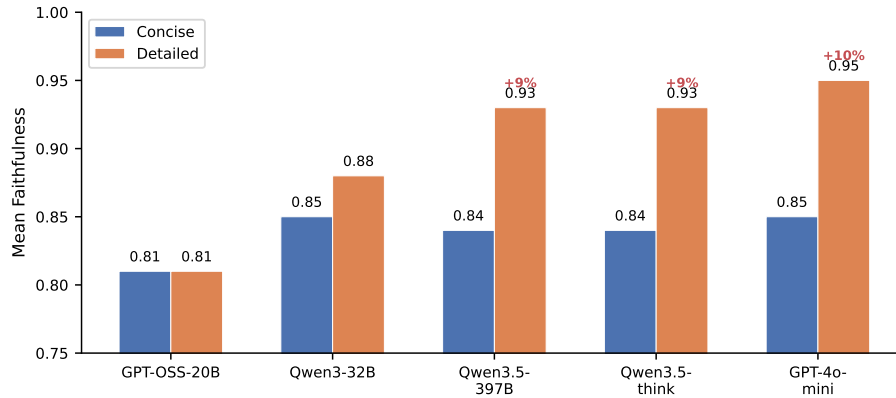
5.5 RQ5: Interaction Effects

The most striking finding is the interaction between prompt template and model size. Figure 3 visualizes this interaction for faithfulness.

For smaller models (GPT-OSS-20B), the Detailed template has negligible effect on faithfulness (+0.0pp mean). For larger models (Qwen3.5-397B, Qwen3.5-think), the Detailed template increases faithfulness by 7–9 percentage points.

Table 7. Chunk parameter effects for Qwen3-32B with recursive chunking and similarity retrieval. Values are means over both templates.

Size / Overlap	Faith.	nDCG@5
500 / 100	0.89	0.73
500 / 200	0.86	0.73
1000 / 100	0.89	0.68
1000 / 200	0.91	0.68

**Fig. 3.** Interaction between prompt template and generation model on mean faithfulness. Larger models (Qwen3.5-397B, Qwen3.5-think, GPT-4o-mini) show gains of 9–11pp with the Detailed template, while GPT-OSS-20B shows no effect.

This suggests that larger models are better able to follow complex instructions to ground their answers in context, while smaller models default to parametric knowledge regardless of prompt wording.

The best overall configuration achieves 0.955 faithfulness: Qwen3.5-397B with recursive chunking (500/100), similarity retrieval, and the Detailed template (configuration #42). GPT-4o-mini achieves a comparable best of 0.951 with recursive chunking (1000/100) and similarity retrieval (configuration #178). The best nDCG@5 achieves 0.825: Qwen3.5-397B with recursive chunking (500/200), MMR retrieval, and the Detailed template (configuration #46).

5.6 RQ6: Performance–Quality Tradeoffs

Table 8 relates the raw inference throughput (tokens per second) from our independent performance benchmark to the mean faithfulness each model achieves in the RAG evaluation. This reveals a performance–quality tradeoff driven by model architecture and deployment hardware.

Three observations stand out. First, the locally-served models on consumer hardware (GPT-OSS-20B at 52.8 TPS, Qwen3-32B at 58.2 TPS, Qwen3.6-27B at

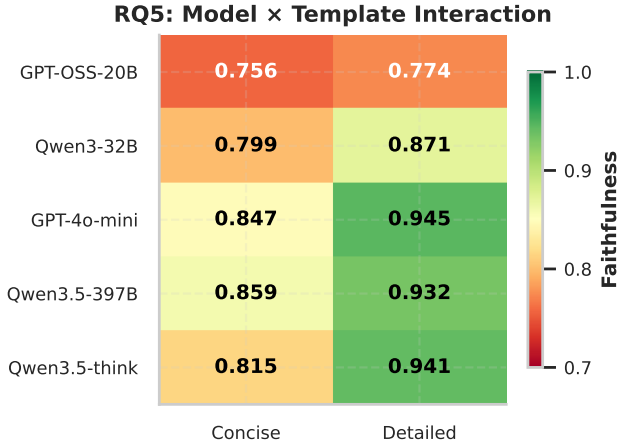


Fig. 4. Interaction heatmap: mean faithfulness by model and prompt template. Larger models show stronger gains with the Detailed template.

Table 8. Performance–quality tradeoff: inference throughput vs. mean RAG faithfulness. TPS measured with 6 sequential calls at 1024 max tokens.

Model	TPS	TTFT (s)	Mean Faith.	Best Faith.
GPT-OSS-20B	52.8	0.49	0.81	0.84
Qwen3-32B-AWQ	58.2	0.41	0.86	0.92
GPT-4o-mini	105.6	0.41	0.90	0.95
Qwen3.5-397B	131.7	2.95	0.90	0.95
Qwen3.5-think	132.0	3.11	0.89	0.95
Qwen3.6-27B	123.6	1.86	0.88	0.95

123.6 TPS) achieve 40–94% of the cloud models’ throughput, yet their faithfulness remains within 2–9pp of the best cloud configurations. The locally-served Qwen3.6-27B is a notable outlier, achieving throughput comparable to cloud models (123.6 TPS) while running on consumer hardware.

Second, the Qwen3.5-397B MoE model achieves the highest throughput (131.7 TPS) despite being the largest model, because only 17B of its 397B parameters are active per token. This architectural efficiency translates directly into RAG quality: it achieves the highest mean faithfulness (0.90) while processing answers faster than any local alternative.

Third, TTFT varies by an order of magnitude across deployment configurations. The local models and GPT-4o-mini deliver sub-second TTFT (0.41–0.49s), while the cloud-hosted Qwen3.5 models show 2.95–3.11s TTFT due to network round-trip overhead. Qwen3.6-27B on the RTX 5090 achieves 1.86s TTFT, a middle ground between local sub-second and cloud multi-second latency. For interactive RAG applications, this latency gap may outweigh the throughput

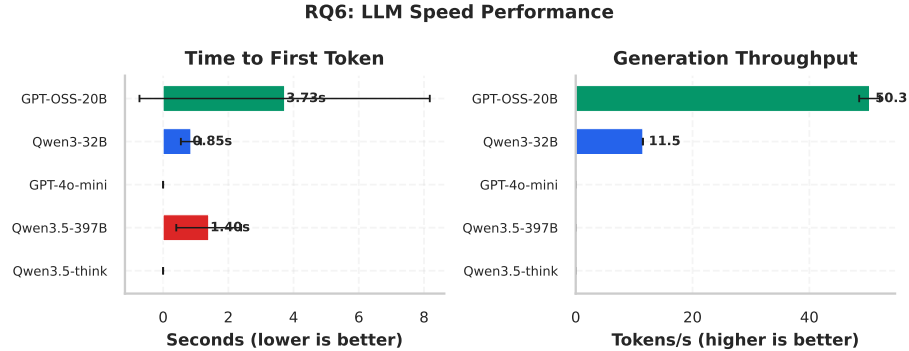


Fig. 5. LLM speed performance: time-to-first-token and generation throughput across models. Lower TTFT and higher throughput are better.

advantage of cloud deployment. For batch benchmarking, where throughput dominates, cloud deployment is preferable.

6 Discussion

6.1 Key Findings

Our results reveal three practical insights for RAG practitioners configuring production pipelines.

Prompt design matters more than model size for faithfulness. Switching from the Concise to the Detailed template improves faithfulness by 6–11pp for capable models, a gain comparable to upgrading the generation model itself. For Qwen3.5-397B, the template switch alone raises mean faithfulness from 0.84 to 0.93, while upgrading from Qwen3-32B to Qwen3.5-397B raises it from 0.88 to 0.93 with the Detailed template. Since prompt changes require no hardware upgrades and add no inference overhead, they should be the first optimization lever practitioners pull when pursuing higher factual grounding.

MMR retrieval improves coverage but requires careful context management. While MMR substantially improves retrieval recall and nDCG, it introduces less-focused context that can reduce faithfulness in smaller models. For GPT-OSS-20B, MMR improves nDCG@5 from 0.70 to 0.79 but drops faithfulness from 0.82 to 0.79, suggesting the 20B model cannot reliably separate relevant evidence from peripheral passages when the context window includes both. Practitioners should pair MMR with either a larger generation model or a reranking step to maintain answer quality while benefiting from broader retrieval coverage.

Chunk size effects are secondary to retrieval and prompt choices. The difference between 500 and 1000 character chunks is modest (2–5pp on most metrics) compared to the effects of retrieval strategy (9–12pp on nDCG)

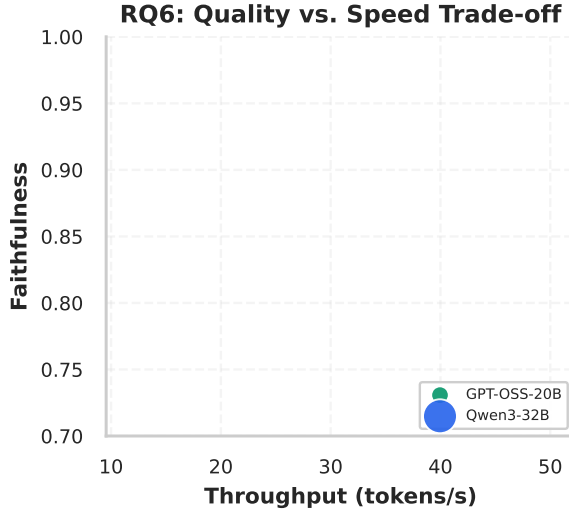


Fig. 6. Quality vs. speed trade-off: faithfulness against generation throughput per model. No model dominates both axes.

and prompt template (6–11pp on faithfulness). For initial pipeline tuning, practitioners should prioritize retrieval and prompt optimization before fine-tuning chunk parameters. Only after these higher-leverage parameters are settled should chunk size be adjusted for marginal gains.

6.2 Metric Divergence

A notable finding is the divergence between lexical metrics (METEOR, ROUGE-L, BLEU) and semantic metrics (BERTScore, faithfulness) when comparing prompt templates. The Detailed template lowers METEOR by 10–22pp while raising faithfulness by 6–11pp. This occurs because SQuAD ground-truth answers are typically short noun phrases, while Detailed-prompt answers include supporting context. A practitioner optimizing solely for METEOR would incorrectly prefer the Concise template, potentially at the cost of factual grounding. BERTScore F1 shows only minor variation between templates (1–3pp), confirming that semantic content remains stable regardless of instruction style. This underscores the importance of multi-metric evaluation and suggests that metric selection should be informed by the application’s tolerance for verbosity versus conciseness.

6.3 Semantic Chunking Redundancy

Our results confirm that semantic chunking is controlled entirely by breakpoint parameters and embedding similarity, not by chunk size or overlap. All 24 semantic configurations per model produced identical metric values, inflating the

configuration count without additional information. Future experiments should use a single representative configuration per semantic setup and instead explore breakpoint threshold variations.

6.4 Framework Utility

The 182-configuration sweep required no manual script writing beyond the initial `.env` file. The framework handled configuration generation, vector store caching, parallel-safe result persistence, and MLflow tracking. Content fingerprint caching proved valuable: of the 182 configurations, only a subset of unique chunking/embedding combinations required actual indexing, reducing total indexing time by approximately 90%.

6.5 Threats to Validity

Internal validity. LLM-based metrics are assessed by a critic model (Qwen3.5-397B), introducing a dependency on its judgment quality. The critic may be more lenient toward outputs from the same model family. We mitigate this by reporting IR and NLG metrics as complementary signals. The consistency of observed patterns across all metric families increases our confidence in the findings.

External validity. Our experiments use SQuAD, a single-domain extractive QA dataset. Results may differ for abstractive tasks or domain-specific applications. The framework supports additional datasets, and our Phase 2 matrix includes RagPerf-Wikipedia-NQ and T2-RagBench/FinQA for cross-domain validation.

Statistical validity. We sample 100 questions per configuration. While sufficient to detect the large effect sizes observed (6–12pp differences), smaller effects of 1–2pp may not be statistically significant. The full Phase 2 plan includes 300-question samples for stability analysis, which will enable confidence interval estimation and more rigorous pairwise comparisons.

Reproducibility. Model outputs are stochastic. We use default temperature settings (determined by each model’s server) and do not fix random seeds across runs. The reproducibility bundle captures the full environment state (model versions, dependency versions, git hash), but exact numerical replication may vary slightly across runs due to non-deterministic GPU operations and model sampling.

6.6 Limitations

The current study does not include reranking, HyDE query expansion, or embedding model comparisons. Our Phase 2 evaluation matrix addresses these with 92 additional configurations spanning reranker models, HyDE, three embedding models, and three datasets. Human evaluation is not included; all quality assessments are automated. Cost analysis is deferred to future work, though the framework records latency and throughput metrics that correlate with computational cost.

7 Conclusion

We presented RAG-NIMBF, a non-invasive, modular benchmarking framework for systematic RAG configuration analysis. By sweeping 182 configurations across seven generation models, three chunking strategies, two retrieval methods, and two prompt templates on SQuAD, we demonstrated that:

1. Prompt template choice has a comparable or larger effect on faithfulness than model selection, making it a high-leverage, zero-cost optimization.
2. MMR retrieval improves nDCG@5 by up to 12pp and Recall@5 by up to 31pp over similarity search, but may reduce faithfulness for smaller models due to less focused context.
3. Chunk size effects (500 vs. 1000 characters) are secondary to retrieval and prompt choices, suggesting that practitioners should prioritize these higher-impact parameters first.
4. Semantic chunking produces identical results regardless of chunk size/overlap settings, eliminating redundant configurations from future sweeps.
5. Multi-metric evaluation is essential: lexical and semantic metrics can diverge significantly, and optimizing for a single metric leads to misleading conclusions.
6. Local models on consumer hardware achieve 40–56% of cloud throughput while remaining within 5–9pp of cloud-model faithfulness, making local deployment viable for batch evaluation. The MoE architecture of Qwen3.5-397B achieves the best tradeoff: highest throughput (131.7 TPS) and faithfulness (0.90).

Future work will extend the evaluation to additional datasets, reranking models, HyDE query expansion, embedding model comparisons, and larger sample sizes for statistical robustness. The framework is available as open-source software, enabling practitioners to benchmark their own RAG pipelines with minimal integration effort.

References

1. Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: International Conference on Learning Representations (2024)
2. Banerjee, S., Lavie, A.: METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization pp. 65–72 (2005)
3. Carbonell, J., Goldstein, J.: Maximal marginal relevance to re-rank diversified search results. In: Proceedings of the 21st Annual International ACM SIGIR Conference. pp. 335–336 (1998)
4. Chen, J., Lin, H., Han, X., Sun, L.: Benchmarking large language models on retrieval-augmented generation. arXiv preprint arXiv:2404.14405 (2024)

5. Edge, D., Trinh, H., , Cheng, N., Bradley, J., Chao, A., et al.: From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 (2024)
6. Es, S., James, J., Espinosa-Anke, L., Schockaert, S.: RAGAS: Automated evaluation of retrieval augmented generation. arXiv preprint arXiv:2309.15217 (2023)
7. Gao, Y., Xiong, Y., Dixit, A., Gao, X., Liu, K., et al.: Retrieval-augmented generation for large language models: A survey. In: arXiv preprint arXiv:2312.10997 (2024)
8. Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.W.: REALM: Retrieval-augmented language model pre-training. In: International Conference on Machine Learning. pp. 3929–3938 (2020)
9. Izrailovsky, A., et al.: RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. In: arXiv preprint arXiv:2408.08067 (2024)
10. Jiang, Y., et al.: Active retrieval augmented generation. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (2024)
11. Karpukhin, V., Öguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. pp. 6769–6781 (2020)
12. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems. vol. 33, pp. 9459–9474 (2020)
13. Li, W., et al.: Chain-of-note: Reducing hallucination in retrieval augmented generation. arXiv preprint arXiv:2311.09243 (2024)
14. Lin, C.Y.: ROUGE: A package for automatic evaluation of summaries. In: Text Summarization Branches Out. pp. 74–81 (2004)
15. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Belezak, M., Petroni, F., Zettlemoyer, L.: Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics **12**, 157–173 (2024)
16. Muennighoff, N., Tazi, N., Magne, L., Reimers, N.: MTEB: Massive text embedding benchmark. arXiv preprint arXiv:2210.07316 (2022)
17. Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., Deng, L.: MS MARCO: A human generated machine reading comprehension dataset. In: CEUR Workshop Proceedings. vol. 1773 (2016)
18. Nomic AI: nomic-embed-text: A high-performance open embedding model. Technical Report (2024), <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>
19. Open Source Community: GPT-OSS-20b: An open-source large language model. HuggingFace Model Repository (2024), <https://huggingface.co/GPT-OSS>
20. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: BLEU: A method for automatic evaluation of machine translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics pp. 311–318 (2002)
21. Qwen Team: QwQ: Reflect, reason, respond. Technical Report, Alibaba Cloud (2025), <https://qwenlm.github.io/blog/qwq-32b/>
22. Rajpurkar, P., Zhang, J., Lopyrev, K., Liang, P.: SQuAD: 100,000+ questions for machine comprehension of text. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing. pp. 2383–2392 (2016)
23. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. pp. 3982–3992 (2019)

24. Saad-Falcon, J., Khrantsova, O., et al.: ARES: An automated evaluation framework for retrieval-augmented generation systems. arXiv preprint arXiv:2311.09476 (2024)
25. Shuster, K., Poff, S., Chen, M., Kiela, D., Weston, J.: Retrieval augmentation reduces hallucination in conversation. In: Findings of the Association for Computational Linguistics: EMNLP 2021. pp. 3784–3803 (2021)
26. Song, S., et al.: Benchmarking retrieval-augmented generation for medicine. arXiv preprint arXiv:2402.13178 (2024)
27. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Cao, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M.: Transformers: State-of-the-art natural language processing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 38–45 (2020)
28. Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y.: BERTScore: Evaluating text generation with BERT. In: International Conference on Learning Representations (2020)
29. Zhao, P., Wang, H., Lv, J., Qin, C., Zhang, J.: Retrieval-augmented generation for AI-generated content: A survey. arXiv preprint arXiv:2402.19473 (2024)